YENEPOYA UNIVERSITY

HIGH LEVEL DESIGN (HLD) DOCUMENT
ON
Dark Web Monitoring for Agriculture Assets

Student Name: Mohammad Ruwais

Registration No: 23BSCED004

Campus ID: 26099

Industry Mentor: Sumit Kumar Shukla

Guided By: Ms. Thanziha

Document Type	High Level Design (HLD)
Version	1.0
Date	April 2026
Domain	Agriculture / Cybersecurity / Dark Web Monitoring

Table of Contents

1. Introduction 4
1.1 Scope of the Document 4
1.2 Intended Audience 4
1.3 System Overview 4
2. System Design 5
2.1 Application Design 5
2.2 Process Flow 5
2.3 Information Flow 6
2.4 Components Design 6
2.5 Key Design Considerations 7
2.6 API Catalogue 7
3. Data Design 8
3.1 Data Model 8
3.2 Data Access Mechanism 8
3.3 Data Retention Policies 9
3.4 Data Migration 9
4. Interfaces 9
5. State and Session Management 10
6. Caching 10
7. Non-Functional Requirements 10
7.1 Security Aspects 10
7.2 Performance Aspects 11
8. References 11

1. Introduction

1.1 Scope of the Document

This High Level Design (HLD) document describes the overall system architecture, design approach, and technology decisions for the Dark Web Monitoring System for Agriculture Assets. It covers the end-to-end workflow from keyword-based threat scanning through dashboard visualization and alert generation. This document serves as the primary architectural reference for development, testing, and deployment activities related to the Agri-Shield Dark Web Monitor.

1.2 Intended Audience

- Software Architects and System Designers
- Backend and Frontend Developers
- Cybersecurity Researchers and Analysts
- Agriculture Sector IT Administrators
- Project Managers and Technical Leads
- Academic Reviewers and Industry Mentors

1.3 System Overview

The Dark Web Monitoring System for Agriculture Assets is a web-based security intelligence platform designed to detect, analyze, and visualize threats targeting the agriculture sector. The system employs a multi-layered architecture:

- Authentication Layer: Secure login mechanism to restrict access to authorized personnel only
- Threat Scanning Layer: Keyword-based log analysis engine that scans input data for agriculture-specific threat indicators such as 'farm database', 'agriculture hack', 'crop data leak', 'farm iot breach', and 'agriculture credentials'
- Analytics Layer: Statistical threat analysis with severity scoring and trend visualization using Plotly charts
- Application Layer: Streamlit-based dashboard rendering live threat reports, severity graphs, and global attack trend analysis

The system provides a role-protected dashboard accessible only after authentication, with three core modules: Monitor Home (real-time scan), Global Threat Analysis (monthly trends), and Project Documentation.

2. System Design

2.1 Application Design

2.1.1 Layered Architecture

The system follows a four-layer architecture that cleanly separates authentication, scanning, analytics, and presentation:

Layer	Component	Technology	Responsibility
L1 – Auth	Login Module	Streamlit session_state	Username/password validation; session persistence
L2 – Scan	Threat Scanner	Python string matching	Keyword detection in log input; severity scoring
L3 – Analytics	Threat Analyzer	Pandas, Plotly Express	Bar charts for detected threats; severity visualization
L4 – App	Dashboard UI	Streamlit, Plotly	Multi-page UI; threat display; global statistics

2.2 Process Flow

The end-to-end data flow follows a pipeline from user authentication through log analysis and threat reporting:

- Step 1 – Login: User submits credentials via the secure login form
- Step 2 – Authentication Check: System validates against credentials; sets session_state['logged_in'] = True on success
- Step 3 – Navigation: Authenticated user selects a module from the sidebar (Monitor Home, Global Threat Analysis, Project Documentation)
- Step 4 – Log Input: In Monitor Home, user pastes raw log data or threat intelligence text into the text area
- Step 5 – Threat Scan: On clicking 'Initiate AI Scan', the system searches input text for predefined agriculture-sector keywords
- Step 6 – Severity Assignment: Each detected keyword is assigned a severity score (1–10 scale)
- Step 7 – Visualization: Detected threats and severity scores are rendered as an interactive Plotly bar chart
- Step 8 – Global Analysis: The Global Threat Analysis module displays a monthly attack trend line chart

2.3 Information Flow

Data flows from user-supplied log input through the keyword detection engine to the visualization layer. There is no external API call or database write in the current version — all processing is in-memory within the Streamlit session.

Data Source	Description
User Log Input	Free-text pasted by authenticated user in the Monitor Home text area
Keyword Engine	Python list: ['farm database', 'agriculture hack', 'crop data leak', 'farm iot breach', 'agriculture credentials']
Threat Output	List of matched keywords with pre-assigned severity levels [9, 8, 7, 10, 6]
Global Stats	Static DataFrame: Jan=45, Feb=62, Mar=89 attacks (demo data)

2.4 Components Design

2.4.1 Component Overview

Component	File / Module	Technology	Description
Login Module	app.py – login()	Streamlit	Renders login form; validates credentials; manages session state
Dashboard Shell	app.py – main block	Streamlit sidebar	Navigation menu; logout button; module routing
Threat Scanner	app.py – Monitor Home	Python, Pandas	Keyword search engine; severity mapping; threat DataFrame builder
Threat Visualizer	app.py – Plotly chart	Plotly Express	Bar chart of detected threats vs severity level
Global Analytics	app.py – Global Threat	Plotly Express	Monthly attack trend line chart with static demo data

2.5 Key Design Considerations

2.5.1 Keyword-Based Detection Approach

Threat detection is implemented as a deterministic keyword-matching engine rather than a probabilistic ML model. This provides:

- Transparency: Every alert can be traced to a specific matched keyword, enabling explainability for security analysts
- Speed: String matching completes in microseconds regardless of log size, with no model inference overhead
- Extensibility: The keyword list can be expanded without retraining — add keywords, add detection capability

2.5.2 Severity Scoring Logic

Each agriculture-sector keyword is mapped to a pre-defined severity score: 'farm database' = 9, 'agriculture hack' = 8, 'crop data leak' = 7, 'farm iot breach' = 10, 'agriculture credentials' = 6. CRITICAL threshold is severity >= 8; WARNING threshold is severity >= 6.

2.5.3 Session-Based Authentication

Streamlit's session_state dictionary provides lightweight in-process session management. The login flag (logged_in) persists across page re-renders within the same browser session. On logout, the flag is reset and the page is re-run to force the login screen.

2.6 API Catalogue

The current version does not expose or consume any external REST APIs. All processing is local within the Streamlit application process. The following internal function interfaces are defined:

Interface	Module	Input	Output
login()	app.py	None (reads st widgets)	Sets session_state['logged_in']
Scan Trigger	app.py Monitor Home	user_input (str)	found_threats (list), chart_data (DataFrame)
Global Stats	app.py Global Threat	None	Plotly line chart rendered to UI

3. Data Design

3.1 Data Model

The system uses in-memory data structures only. No persistent database is employed in the current version. The primary data model used during a scan session is:

Field	Type	Example Value	Description
Threat Found	String	'farm iot breach'	Matched keyword from the agriculture threat dictionary
Severity Level	Integer	10	Pre-assigned threat severity score (1–10 scale)
Scan Timestamp	String	Session time	Time of scan initiation (session-scoped, not persisted)
Month	String	'Jan', 'Feb', 'Mar'	Time axis for global trend analytics (static demo data)
Attacks	Integer	45, 62, 89	Number of recorded agriculture sector attacks per month

3.2 Data Access Mechanism

All data access is in-memory via Python variables and Pandas DataFrames. Threat scan results are constructed as a DataFrame on-the-fly after each scan and immediately passed to the Plotly visualization engine. No read/write operations to disk or external databases are performed. Global trend data is initialized as a static DataFrame at the time the Global Threat Analysis module loads.

3.3 Data Retention Policies

In the current version, no data is retained between sessions. All scan results are ephemeral — they exist only in Streamlit's in-memory state during the active browser session. Refreshing the page or logging out discards all scan history. Future versions should implement a logging backend (e.g., SQLite or Firebase) to persist scan history for audit trails and trend analysis.

3.4 Data Migration

No formal data migration is required for the current version since the system uses no persistent storage. When a database backend is introduced in a future version, a migration script will be required to import any historically collected CSV logs into the new schema.

4. Interfaces

4.1 Software Interfaces

Interface	Module	Description
Streamlit UI	app.py – all modules	Web browser UI at localhost:8501; handles all user interaction
Plotly Charts	app.py – px.bar(), px.line()	Interactive threat visualization; hover tooltips; zoom/pan
Pandas DataFrames	app.py – pd.DataFrame()	Internal data structure for threat results and global stats

4.2 User Interface

The Streamlit dashboard provides a web-based UI accessible at localhost:8501. Key UI elements include:

- Login Screen: Username/password form with error feedback on invalid credentials
- Sidebar Navigation: Module selector dropdown (Monitor Home, Global Threat Analysis, Project Documentation) and Logout button
- Monitor Home: Text area for log input; 'Initiate AI Scan' button; threat detection results with severity bar chart

- Global Threat Analysis: Monthly attack trend line chart with interactive Plotly controls
- Project Documentation: Static information page describing the system purpose

5. State and Session Management

5.1 Session State Variables

Streamlit's session_state dictionary maintains the following state across page re-renders within a single browser session:

- logged_in (bool): Tracks whether the current user has successfully authenticated. Initialized to False on first load; set to True on successful login; reset to False on logout.

No additional server-side session persistence is maintained. All other variables (scan results, chart data) are recomputed on each Streamlit re-run triggered by user interaction.

6. Caching

The current version does not implement explicit caching. Streamlit rerenders the entire application state on each user interaction. Future enhancements may include:

- @st.cache_data decorator for the global statistics DataFrame to prevent recomputation on every re-run
- In-memory scan history cache (Python deque) to accumulate past scan results within a session for trend analysis
- Redis or session-scoped cache for storing the keyword dictionary and severity mappings if the keyword list grows large

7. Non-Functional Requirements

7.1 Security Aspects

Aspect	Description
Authentication	Login required before accessing any dashboard module; session_state guards all protected routes
Credential Storage	Credentials are currently hardcoded; production deployments must use environment variables or a secrets manager
Input Handling	User log input is processed as plain text; no SQL injection or command injection risk in the current keyword-matching approach

Network	Streamlit serves over HTTP by default; HTTPS/TLS must be configured in production via a reverse proxy
Access Control	No role-based access control in current version; all authenticated users have full access to all modules

7.2 Performance Aspects

Parameter	Value / Description
Scan Latency	< 10ms for keyword matching on typical log inputs (pure Python string operations)
Dashboard Load Time	< 2 seconds on local deployment; dependent on Streamlit startup and Plotly rendering
Concurrent Users	Single-user design (Streamlit default); multi-user requires separate Streamlit server instances
Scalability	Keyword list extensible without performance degradation up to ~10,000 keywords; beyond that, Aho-Corasick algorithm recommended
Memory Usage	Minimal — all DataFrames are small (< 1KB); no large in-memory data structures

8. References

Reference	URL / Source
Streamlit Documentation	https://docs.streamlit.io
Plotly Express Python	https://plotly.com/python/plotly-express/
Pandas Documentation	https://pandas.pydata.org/docs/
OWASP Dark Web Monitoring	https://owasp.org/www-community/
Agriculture Cybersecurity – CISA	https://www.cisa.gov/food-and-agriculture-sector
Python Session State	https://docs.streamlit.io/library/api-reference/session-state